

Predecir la Salud de la Batería de una Laptop con Machine Learning

Ximena Pérez Escalante

A01751827

Instituto Tecnológico y de Estudios Superiores de Monterrey

Agosto 2026

Abstract La degradación de la batería de los ordenadores portátiles es un problema común que afecta a los usuarios que dependen de estos equipos para actividades académicas, profesionales y personales. El estado de salud de la batería puede deteriorarse con el tiempo debido a factores como los patrones de uso, los ciclos de carga, el consumo de energía y la temperatura de funcionamiento. Comprender la relación entre estos factores y el estado de la batería ayuda a identificar las condiciones asociadas a una degradación más acelerada. Este estudio tiene como objetivo predecir el estado de salud de la batería de un portátil expresado como porcentaje utilizando múltiples variables relacionadas con la condición de la batería y el uso del equipo. Para desarrollar un modelo de aprendizaje automático, se utilizó un conjunto de datos sintético denominado ‘Laptop Battery Health Prediction Dataset’. Los resultados demuestran que la regresión lineal permite modelar eficazmente el estado de salud de la batería a partir de las variables seleccionadas de uso y estado de la misma, ofreciendo un enfoque útil para comprender y predecir la degradación de la batería.

Keywords): Regresión lineal, salud de batería, machine learning, gradiente descendiente.

Introducción

Una gran parte de la población trabaja con una laptop, ya sea para la escuela, el trabajo o uso personal. Y con cada

laptop se vive el mismo problema, la degradación de la batería. De acuerdo al uso que se le dé, depende la rapidez con la que esto ocurra.

Por eso es común ver a personas que cuando usan su laptop, nunca la desconectan de la corriente. Y al preguntarles por qué hacen esto, su respuesta es que se le acaba muy rápido la batería, por lo que es más fácil dejarla conectada. Pero aunque eso ayuda en el momento, termina empeorando la salud de la misma.

Hoy en día, la mayoría de las laptops tienen baterías de iones de litio, cuyas características son que se cargan rápidamente, se descargan a un ritmo constante y poseen una alta densidad energética que permite utilizar celdas pequeñas. De igual manera, la capacidad de las celdas de iones de litio se reduce después de un cierto número de veces en que la batería se carga y posteriormente descarga.

Existen múltiples artículos donde se describen maneras en las que se puede alentar el proceso de degradación de la batería. Entre ellas se encuentran:

- Evitar las descargas profundas frecuentes
- Mantener la batería dentro de un rango de aproximadamente el 20% y 80%
- No usar o cargar la laptop a altas temperaturas.

- Evitar almacenar la laptop completamente cargada o completamente descargada.

A pesar de que se documenten estas alternativas, comúnmente, los usuarios suelen tener malos hábitos de uso, por lo que se busca conocer cómo impactan directamente distintas variables, la salud de la batería para saber específicamente cómo preservar la salud de la misma.

A partir de esto, se planteó el objetivo de predecir la salud de la batería (battery health) medida en porcentaje, a partir de diferentes variables descritas en la siguiente sección.

Descripción del Dataset

Se utilizó “Laptop Battery Health Prediction Dataset”. Un Dataset sintético que simula el comportamiento de una laptop, modelando los cambios en la batería de la misma dependiendo del uso que se le dé. En total, tiene 4600 instancias y 10 features descritos en la siguiente tabla:

Nombre	Significado	Tipo de dato
--------	-------------	--------------

Battery Age	Edad de la batería medida en años	Integer
Daily Usage Hours	Promedio de horas de uso al día	Decimal
Gaming User	Indica si el usuario juega videojuegos frecuentemente (Sí/No)	Integer
Design Capacity	Capacidad original de la batería cuando era nueva (mAh)	Integer
Cycle Count	Número total de ciclos charge-discharge completados	Integer
CPU Usage	Promedio de utilización del CPU durante uso diario (%)	Decimal
GPU Usage	Promedio de utilización del GPU durante uso diario (%)	Decimal
Power Consumption	Promedio de energía consumido durante la operación (Watts)	Decimal
Average Temperature	Promedio de temperatura de la batería en operación (°C)	Decimal
Full Charge Capacity	Máxima capacidad de carga actualmente después de	Decimal

	degradation	
--	-------------	--

Tabla 1: Descripción de Features del Dataset

La variable que se busca predecir es la siguiente:

Nombre	Significado	Tipo de dato
Battery Health	Salud de la batería medida en porcentaje	Decimal

Tabla 2: Descripción de la variable a predecir del Dataset

Al representar datos en un modelo de Machine Learning, tenemos dos tipos de datos:

- *Númericos*: Battery Age, Daily Usage Hours, Design Capacity, Cycle Count, CPU Usage, GPU Usage, Power Consumption, Average Temperature, Full Charge Capacity.
- *Catégoricos*: Gaming User.

A partir de estos tipos podemos realizar otra división en numérico discreto y continuo:

- *Númericos Discretos*: Battery Age, Design Capacity, Cycle Count.
- *Númericos Continuos*: Daily Usage Hours, CPU Usage, GPU Usage, Power Consumption, Average Temperature, Full Charge Capacity.
- *Númerico discreto de clase*: Gaming User.

ETL

ETL, por sus siglas en inglés se refiere al proceso: *Extract - Transform - Load*.

Extracción

Esta fase consiste en obtener los datos que se utilizarán en el modelo. Esto se puede realizar de distintas maneras. Para este trabajo, se extrajo la totalidad de los datos de un Dataset (full-extract), el cual es un archivo de tipo .csv, lo que quiere decir que está en un formato preparado para iniciar el proceso de transformación.

Transformación

Cuando se termina la fase de extracción, lo más probable es encontrar:

- Datos faltantes por columna o filas completas.
- Datos atípicos (outliers).
- Columnas irrelevantes que no proporcionan ningún valor al entendimiento de los datos.
- Filas duplicadas.

Al revisar el Dataset se identificó que no había datos nulos o filas duplicadas.

Lo cual quiere decir que no es necesario eliminar filas o imputar valores.

Análisis Exploratorio de Datos (AED)

Análisis de Correlación entre variables

Continuando con el análisis exploratorio de los datos, se optó por calcular la matriz de correlación de las variables, para después hacer un mapa de calor. Ya que este permite visualizar esto de manera clara, donde el rojo significa correlación positiva y el azul una correlación negativa.

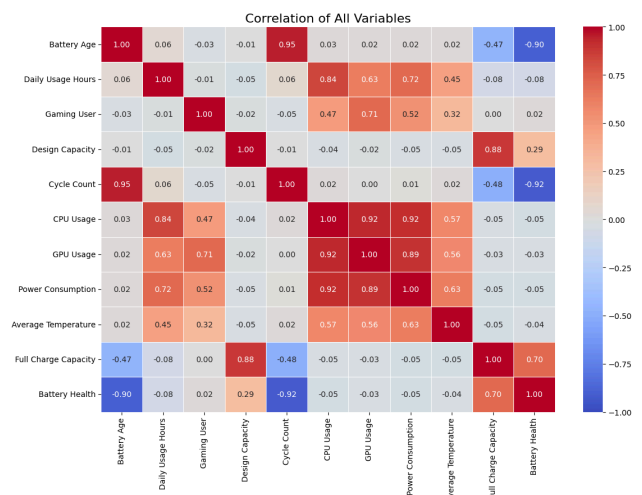


Figura 1: Mapa de Calor de Correlación entre variables

Correlaciones positivas con Battery Health (de mayor a menor):

- Full Charge Capacity (0.70)
- Design Capacity (0.29)

Correlaciones negativas con Battery Health (de mayor a menor):

- Cycle Count (0.92)
- Battery Age (0.90)

En este caso, es necesario poner atención a las relaciones, porque la fórmula para calcular la salud de la batería de una laptop es la siguiente:

$$\text{Battery Health} = \left(\frac{\text{Full Charge Capacity}}{\text{Design Capacity}} \right)$$

Por lo que al ver que estas dos variables son las que muestran la mayor relación positiva con Battery Health significa que al mantenerlas en el Dataset para entrenar el modelo, puede generar que el mismo sea inservible.

Ya que conocemos esta fórmula y su relación directa con la variable que queremos predecir, es necesario eliminarlas del Dataset.

Valores atípicos (outliers)

Encontrar valores atípicos puede sesgar los datos, perjudicando las predicciones que genera el modelo. Sin embargo, en otras ocasiones esto puede ser de utilidad para predecir datos reales.

Para identificar estos valores en el Dataset, se generaron Diagramas de caja y bigotes (Boxplots) por cada feature. A continuación se muestran únicamente las variables que cuentan con valores atípicos:

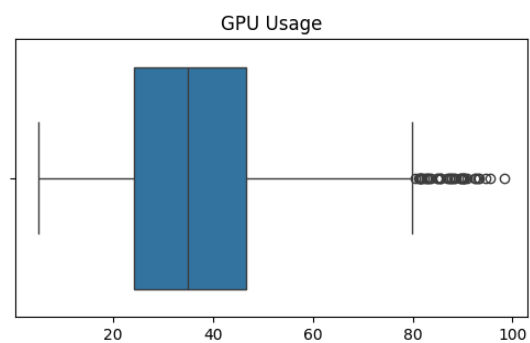


Figura 2: GPU Boxplot

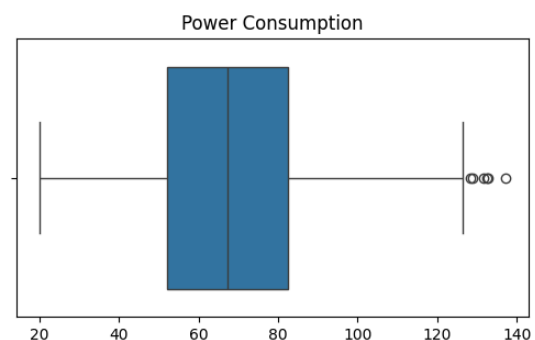


Figura 3: Power Consumption Boxplot

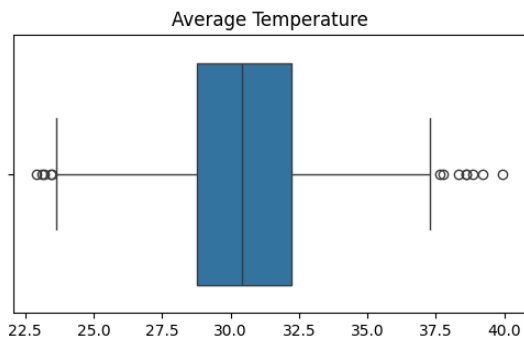


Figura 4: Average Temperature Boxplot

Debido al contexto del proyecto, donde se busca entender cómo cada variable impacta en la degradación de la batería de una laptop, se decidió no eliminar las instancias con valores atípicos de cada columna. Esto con el fin no sólo de mantener los datos realistas y preparar el modelo para estos casos, sino porque estos valores no son errores de mediciones, reiterando que es importante conservar dichos escenarios realistas.

Escalamiento de los datos

En este Dataset los features (excluyendo la variable categórica) tienen diferentes rangos numéricos, lo cual puede afectar de manera el modelo. Por ello, es necesario escalar las variables numéricas continuas.

División del Dataset en Entrenamiento y Prueba

Es importante mencionar que antes de realizar el escalamiento, se debe separar el Dataset en features (x) y target (y). Posteriormente se dividen ambos en entrenamiento (80%) y prueba (20%).

Esto con el objetivo de evitar el filtrado de datos.

Una vez realizado esto, se utilizó la estandarización como método de escalamiento, definido por la siguiente fórmula:

$$X_{estandarizada} = \frac{X - promedio(X)}{std(X)}$$

La razón por la que se eligió este método es que es comúnmente utilizado para regresiones lineales, por lo que se ajusta a la necesidad de este trabajo.

Carga de datos (Loading)

Para esta etapa simplemente se cargó el Dataset con las modificaciones que fueron realizadas anteriormente.

Marcando el final del proceso de ETL.

Modelo de Regresión Lineal v1.0

Al contar con un problema donde el objetivo es predecir una variable numérica, las predicciones se deben realizar con un modelo de regresión lineal, cuya fórmula es:

$$y = mx + b$$

Donde m (pendiente) y b (bias) son los parámetros.

Este modelo está conformado por tres funciones: hipótesis, costo y optimización. Las cuales se describirán a continuación.

Función de hipótesis

Es la función de regresión lineal inicial, la cual no ha sido comprobada. Para saber cuál es el mejor modelo, se comparan múltiples hipótesis.

Función de costo

Permite saber qué tan bien se ajusta el modelo a los datos. Es decir, compara la distancia entre la predicción del modelo y el valor real. En este caso se utilizará el Mean Squared Error, cuya fórmula es:

$$\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Función de optimización

Como su nombre lo indica, es la que optimiza el modelo para disminuir el costo, encontrando el valor ideal de m .

Como entrada tiene la función de costo, que ya incluye la función de hipótesis.

Para realizar esto, se utiliza la gradiente descendiente, que calcula la derivada de la pendiente, lo que permite saber si el valor de m debe aumentar o disminuir para llegar a su valor ideal.

Esto se expresa de la siguiente manera:

$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

Donde θ = parámetros a optimizar;

α = tasa de aprendizaje; J = función de costo.

Resultados

Se corrió la función de optimización por 3000 epochs con un learning rate (ALPHA) de 0.35.

Los parámetros finales quedaron de la siguiente manera:

- Sesgo (theta_0) 83.1042
- Battery Age -2.2381

- Daily Usage Hours
-0.4671
- Gaming User -2.6641
- Cycle Count -7.0532
- CPU Usage -0.0274
- GPU Usage 1.4867
- Power Consumption -0.8200
- Average Temperature -0.1048

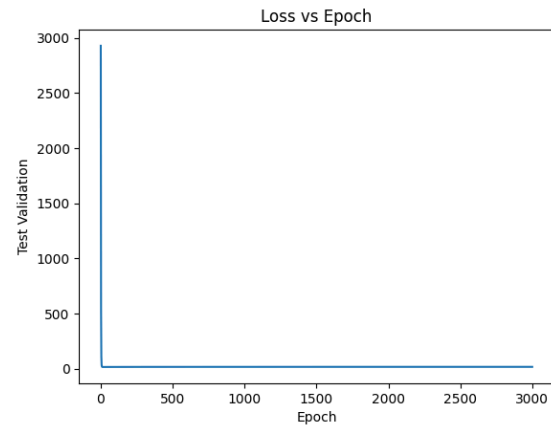


Figura 5: Pérdida en test

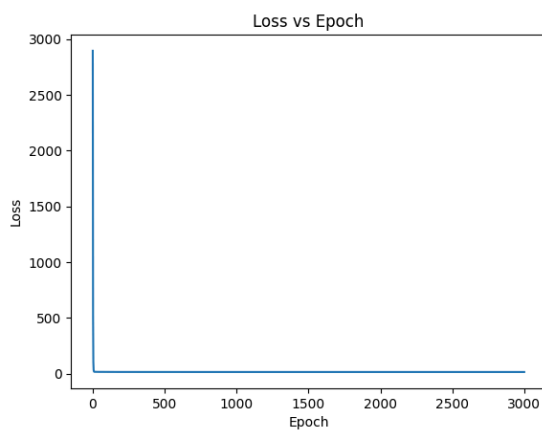


Figura 5: Pérdida en train

Graficando un scatter plot, es más fácil observar el desempeño del modelo:

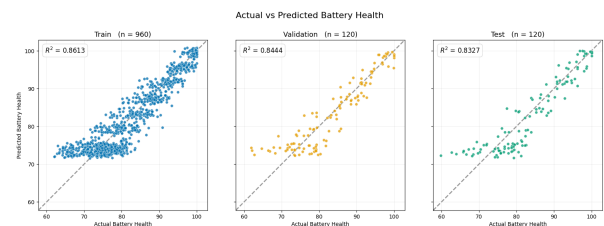


Figura 6: Scatterplot predicciones

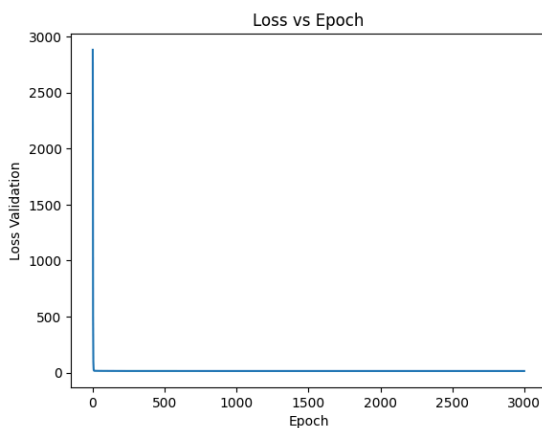


Figura 5: Pérdida en validation

Split	R-squared	Loss
Train	0.8613	13.7031
Validation	0.8444	14.7236
Test	0.8327	16.5174

Tabla 3: Valores de R-squared y pérdida por split

Diagnóstico

Al observar los resultados, se pueden apreciar los siguientes puntos:

- La pérdida en cada split es muy similar, con sólo más o menos uno o dos puntos de aumento en cada split.
- Los valores de R-squared, la cual indica qué porcentaje de la variabilidad de los datos de cada split es explicada por el modelo, disminuye uno o dos puntos en cada split.
- El scatterplot, a simple vista, muestra que el modelo en su mayoría sí está prediciendo correctamente en todas las etapas.
- Los valores de R-squared mayor al 80% dan indicio de que no existe un bias alto, es decir, no hay underfitting.
- La varianza entre train y val (0.0169), tanto como la de train con respecto a test (0.0286) son señal de que no existe overfitting de igual manera, es decir, el modelo no se entrenó a los datos de entrenamiento.

Mejora

Regresando a la figura 6, hay un detalle peculiar, a pesar de que la línea pasa en medio de múltiples puntos, se puede apreciar una ligera curva, que siempre deja muchos valores alejados de la línea. Esto es un indicio de que este no es un problema lineal.

Uno de los trucos que se suelen utilizar en estos casos para no modificar el modelo que se está utilizando es transformar los datos, ya sea llevarlos al cuadrado, sacar su raíz cuadrada, logaritmo, entre otros.

Al hacer pruebas transformando diferentes features y comparando los resultados podremos saber cuál es el camino que debemos tomar para mejorar el modelo.

En este caso se usará para las variables numéricas discretas (Battery Age y Cycle Count), ya que aunque hayan sido estandarizadas, es posible que su tipo de dato esté ocasionando esta no linealidad.

Bibliografía

Dharmendra Pandit. (s. f.). *Laptop battery health dataset* [Data set].

Kaggle.

<https://www.kaggle.com/datasets/dharmendrapandit12/laptop-battery-health-dataset>

Microsoft. (s. f.). *Caring for your battery in Windows*. Microsoft Support.
[Caring for your battery in Windows](#)